Lecture 26: Scratch Implementations — Problem Set (Questions)

This problem set tests your first-principles comprehension of linear, logistic, and metric-space machine learning algorithms. Implement every algorithm using pure Python and NumPy without relying on Scikit-Learn model classes.

Problem 01: Vectorized Batch Gradient Descent Linear Regression

Objective

Design and implement a clean, vectorized `LinearRegressionGD` class from scratch using pure Python and NumPy.

Requirements

1. Implement `__init__(self, lr=0.01, n_iter=1000)` to store optimizer hyperparameters.
2. Implement `fit(self, X, y)`:
   - Initialize weights $\mathbf{w} = \mathbf{0}_d$ and bias $b = 0$.
   - Compute predictions $\hat{\mathbf{y}} = X\mathbf{w} + b$.
   - Compute the analytical gradients with respect to $\mathbf{w}$ and b:

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y})$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

   - Update parameters per iteration: $\mathbf{w} := \mathbf{w} - \alpha \nabla_{\mathbf{w}} J, b := b - \alpha \frac{\partial J}{\partial b}$.
   - Store cost $J(\mathbf{w}, b)$ in a `loss_history` list to verify monotonic convergence.
3. Implement `predict(self, X)` to generate predictions on unseen test feature matrices.

Problem 02: Analytical Normal Equation with L2 Ridge Regularization

Objective

Implement `LinearRegressionRidgeOLS`, solving linear regression via the closed-form Normal Equation with an L2 ridge penalty to prevent numerical instability from singular matrices.

Requirements

1. Augment the design matrix $X \in \mathbb{R}^{m \times d}$ with an intercept bias column: $X_b = [\mathbf{1}_m, X] \in \mathbb{R}^{m \times (d+1)}$.
2. Formulate the L2-regularized Normal Equation:

$$\boldsymbol{\theta} = \left(X_b^T X_b + \lambda I'\right)^{-1} X_b^T \mathbf{y}$$

Where $I' \in \mathbb{R}^{(d+1)\times(d+1)}$ is the identity matrix with $I'_{0,0} = 0$ (the intercept is never penalized). 3. Extract $\text{bias} = \theta_0$ and $\text{weights} = \theta_{1:}$. 4. Implement `predict(self, X)` returning $X\mathbf{w} + b$.

Problem 03: Vectorized Logistic Regression with Numerical Safeguards

Objective

Implement `LogisticRegressionL2` for binary classification with vectorized gradient descent, probability estimation, and L2 regularization.

Requirements

1. Implement `_sigmoid(self, z)`:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

2. In `fit(self, X, y)`:
   - Compute probability predictions: $\hat{\mathbf{y}} = \sigma(X\mathbf{w} + b)$.
   - Compute L2-regularized gradients:

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y}) + \frac{\lambda}{m} \mathbf{w}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

- Update $\mathbf{w}$ and b using gradient descent.

3. Implement `get_probabilities(self, X)` and `predict(self, X, threshold=0.5)`.

Problem 04: Multi-Class Softmax Regression from Scratch

Objective

Extend logistic regression to K-class categorical classification by implementing `SoftmaxRegression`.

Requirements

1. Given feature vector $\mathbf{x} \in \mathbb{R}^d$ and weight matrix $W \in \mathbb{R}^{d \times K}$, compute logits $\mathbf{z} = W^T \mathbf{x} + \mathbf{b} \in \mathbb{R}^K$.
2. Implement numerically stable Softmax using max-subtraction to prevent floating-point overflow:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(z_k - \max(\mathbf{z}))}{\sum_{j=1}^{K} \exp(z_j - \max(\mathbf{z}))}$$

3. Compute the Categorical Cross-Entropy gradient:

$$\nabla_W J = \frac{1}{m} X^T (\hat{Y} - Y)$$

Where $Y \in \mathbb{R}^{m \times K}$ is the one-hot encoded label matrix.

Problem 05: Generalized K-Nearest Neighbors with Minkowski Metrics

Objective

Build a versatile non-parametric classifier `KNNClassifierAdvanced` supporting Euclidean (L_2), Manhattan (L_1), and arbitrary Minkowski (L_p) distance metrics.

Requirements

1. Support metric parameter:

$$D_p(\mathbf{x}_1, \mathbf{x}_2) = \left(\sum_{j=1}^{d} |x_{1j} - x_{2j}|^p \right)^{1/p}$$

2. Support weighting modes:
   - Uniform voting: each neighbor casts an equal vote of 1.
   - Distance-weighted voting: neighbor i casts vote $w_i = \frac{1}{d_i + 10^{-6}}$.
3. Plurality vote tallying returning predicted discrete classes.

Problem 06: K-Nearest Neighbors Continuous Regressor

Objective

Implement `KNNRegressor` for non-parametric continuous target estimation.

Requirements

1. Store training points (X, y) in $\mathcal{O}(1)$ time.
2. Given a query point $\mathbf{x}$, retrieve the k nearest neighbors $\mathcal{N}_k(\mathbf{x})$.
3. Predict target as the weighted average:

$$\hat{y}(\mathbf{x}) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i y^{(i)}}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i}$$

Problem 07: Mini-Batch & Stochastic Gradient Descent Optimizer

Objective

Design a modular optimizer engine `MiniBatchGDOptimizer` capable of executing:

1. Batch Gradient Descent (Batch size $B = m$).
2. Stochastic Gradient Descent (Batch size $B = 1$).
3. Mini-Batch Gradient Descent ($1 < B < m$).

Requirements

- Randomly shuffle training data at each epoch.
- Slice data into batches of size B and update parameters on each mini-batch.
- Benchmark parameter trajectory stability and runtime across the three modes.

Problem 08: End-to-End Parity Test Harness vs Scikit-Learn

Objective

Develop a production test suite that evaluates scratch implementations against Scikit-Learn models on identical datasets.

Requirements

1. Train `LinearRegressionGD` and `LinearRegressionOLS` vs `sklearn.linear_model.LinearRegression`. Assert coefficient parity $\|\mathbf{w}_{\text{scratch}} - \mathbf{w}_{\text{sklearn}}\| < 10^{-4}$.
2. Train `LogisticRegressionL2` vs `sklearn.linear_model.LogisticRegression`. Assert test accuracy parity $> 98\%$.
3. Train `KNNClassifierAdvanced` vs `sklearn.neighbors.KNeighborsClassifier`. Assert exact prediction equality.

Systems Engineering Case Studies

- **Case Study 1**: Loss Surface Geometry, Hessian Condition Number & Learning Rate Dynamics in Gradient Descent.
- **Case Study 2**: The Curse of Dimensionality & Asymptotic Complexity — OLS vs Gradient Descent.

- **Case Study 3**: Metric-Space Geometry, Voronoi Tessellations & High-Dimensional Distance Concentration.